

Base de conocimiento vectorial

CODEFEST AD ASTRA 2026 - Etapa 1 · Documento técnico

Equipo **S.T.A.R.S** - Luis Mendoza (líder), Deiner González, Valeria Berrio, Mark Pastrana. Universidad de los Andes · Fuerza Aeroespacial Colombiana · 13 de agosto de 2026

1. Alcance

Este documento describe la base de conocimiento vectorial construida sobre el corpus de fuentes abiertas provisto por ADL y el módulo de recuperación asociado. Cubre los cuatro puntos que exige §1.4: estrategia de fragmentación y su justificación, encoder seleccionado y criterios de elección, tipo de índice FAISS empleado y descripción del grafo de conocimiento.

El sistema indexa la totalidad del corpus de los tres fenómenos, conforme a §1.3: 1.826 archivos distribuidos en F1 (459), F2 (479) y F3 (888). Ninguna etapa emplea modelos generativos. La extracción y la normalización son procesamiento determinista de texto; la representación vectorial procede de un encoder de la familia BERT; y la recuperación opera exclusivamente sobre vectores, puntuaciones de similitud y metadata, conforme a §4.2 y §8.3.

El procesamiento se organiza en seis etapas encadenadas, cada una con salida persistente en disco, lo que permite reejecutar cualquiera sin repetir las anteriores. El coste no es simétrico entre ellas y esa asimetría condicionó varias decisiones de diseño.

Etapas	Salida	Volumen	Coste
1. Extracción (§2.1)	documentos con texto crudo	1.826	4 h 09 min
2. Normalización (§2.2)	documentos limpios, con idioma	1.826	4,5 min
3. Fragmentación (§3)	fragmentos	91.021	24 s
4. Codificación (§4)	matriz de vectores	91.021 x 1.024	2 h 48 min (GPU T4)
5. Indexación (§5)	índice FAISS y almacén de metadata	372,8 MB + 240,1 MB	< 1 min
6. Recuperación (§8, §9)	archivo de resultados	50 líneas	34 s

Invariante de orden. La fila i de la matriz de vectores corresponde a la línea i del archivo de fragmentos y a la línea i del `metadata.jsonl`, que a su vez es el identificador interno i del índice FAISS. Este invariante materializa el requisito de §5.3. Un desalineamiento en esa cadena no produciría ninguna excepción: devolvería la metadata de un fragmento distinto al recuperado, con resultados plausibles y erróneos. Por eso cada registro de metadata incluye un campo `faiss_id` con su número de línea y el invariante se verifica de forma explícita.

2. Preprocesamiento de las fuentes

2.1 Extracción y enrutamiento

El corpus es heterogéneo: 954 JSON, 759 PDF, 73 PBF, 26 CSV, 8 JPG, 4 XLSX, 1 AVIF y 1 TXT. Se implementó un extractor por familia de formato, todos con la misma interfaz, y un orquestador que enruta cada archivo según su extensión real.

El inventario en Excel provisto por ADL es la autoridad sobre qué constituye un documento y sobre su identificador. El `doc_id` se toma de él en lugar de generarse, decisión que el comité organizador confirmó posteriormente como requisito: el emparejamiento con el conjunto de referencia se realiza mediante el `DOC_ID` oficial. El campo `formato` registra la extensión real del archivo en minúsculas, siguiendo asimismo la aclaración oficial de que la enumeración de la Tabla 1 es ilustrativa y no exhaustiva.

La única excepción a la autoridad del inventario es la verificación por número mágico. Dos archivos con extensión `.pdf` comienzan por `<!DOCTYPE html>` y no son PDF. El orquestador comprueba la firma `%PDF-` y, cuando falta, corrige el formato a HTML y enruta el archivo al extractor correspondiente. No es una heurística: un archivo que no lleva su propia firma se desmiente a sí mismo. El corpus no se modifica en disco en ningún caso.

Para los archivos JSON se emplea selección explícita de campos y no un recorrido genérico del árbol. El corpus contiene seis esquemas distintos, y un recorrido genérico incorporaría al texto indexado campos como `url`, `authors` o `pdf_links`, que no son contenido del documento. Se verificó que `body_text` y `body_paragraphs` son idénticos en los 229 archivos donde ambos aparecen, por lo que nunca se concatenan.

2.2 Reconocimiento óptico de caracteres

§2.1 recomienda aplicar OCR sobre imágenes con texto relevante. El OCR clásico no es un modelo generativo, de modo que su uso no incurre en la prohibición de §4.2. Se adoptaron dos criterios opuestos, ambos decididos con medición.

OCR de página completa: activado. Cincuenta y un PDF del corpus son documentos escaneados sin capa de texto y producían cero caracteres. Cuando un documento completo rinde menos de 200 caracteres extraíbles, cada página se rasteriza y se somete a OCR. Los 51 pasaron de cero a una mediana de 5.310 palabras, ninguno por debajo de 313. La resolución se fijó en 150 DPI midiendo la confianza que reporta el propio motor sobre once páginas de cuatro documentos en español e inglés: 150 DPI obtiene 91,95 de confianza media y 378 caracteres acentuados en 52,8 s, frente a 91,34 y 343 en 81,6 s con 300 DPI, que resulta simultáneamente peor y un 55 % más lento. La confianza del motor es mejor criterio que el recuento de palabras, porque más palabras puede significar más ruido.

OCR de figuras: desactivado. Activarlo generaba 1.755 documentos sintéticos adicionales sobre 1.826 archivos, el 49,2 % del índice, y con un reparto que agravaba el desequilibrio del corpus: 851 figuras en F1 y 881 en F2, frente a 23 en F3, que es el fenómeno con más consultas asignadas. Sobre una muestra de 178 figuras de 60 PDF repartidos por todo el corpus, solo el 19 % aportaba texto aprovechable, y ese porcentaje estaba sistemáticamente degradado: la sigla «AI» no aparecía correctamente escrita ni una sola vez, y cuatro archivos la contenían como «Al» con ele minúscula, error clásico de confusión I/l aplicado precisamente al término central del fenómeno 1.

El argumento que cierra la decisión procede de la Tabla 1, que define `texto` como «texto original del fragmento, sin modificaciones». Indexar transcripciones con esa tasa de error introduciría en la base de conocimiento afirmaciones que el documento no contiene. A una gráfica de barras no hay texto que extraerle; a una página escaneada, sí. La funcionalidad se conserva tras un interruptor, no eliminada.

Desactivarlo tiene además una consecuencia valiosa: se mantiene la correspondencia uno a uno entre archivos y documentos, de modo que cada `doc_id` designa un archivo real del inventario de ADL y no un identificador sintético.

Los idiomas de OCR se limitan a español e inglés. Se comprobó que los 51 documentos rescatados son 47 en español y 4 en inglés, ninguno en portugués: las fuentes lusófonas del corpus se entregan en JSON y PBF, no como PDF escaneados.

2.3 Normalización

Se implementaron ocho pasos deterministas: los cuatro que §2.2 requiere -normalización a UTF-8 y forma NFC, eliminación de caracteres de control y espaciado redundante, supresión de elementos repetitivos sin valor informativo, y detección del idioma predominante- y cuatro que el corpus hizo necesarios.

El principio que gobierna esta etapa es que se elimina ruido, nunca información: cada regla que suprime contenido va acompañada de la medición que demuestra que lo suprimido no era contenido del documento. Ese criterio descartó, entre otras, la idea de indexar únicamente las columnas textuales de los archivos tabulares, que habría eliminado identificadores capaces de discriminar un registro.

Paso adicional	Justificación medida
Decodificación de marcadores (<code>cid:N</code>)	18 documentos los contienen en lugar de caracteres, por fuentes empujadas sin tabla ToUnicode, dos de ellos en más del 99 % del texto. Se busca el desplazamiento constante entre índice de glifo y código de carácter y se acepta solo si produce palabras funcionales reales. Un documento pasó de 473 palabras ilegibles a 1.556; otro, de 22.896 a 52.269.
Caracteres de control	El corpus exige comportamientos opuestos: en 21 PDF el carácter <code>\x07</code> ocupa la posición del espacio y eliminarlo une las palabras; en otros 6 aparece dentro de ellas y sustituirlo las parte. Se evalúan ambas variantes por documento y se conserva la que produce más palabras plausibles. La función es pura y por tanto determinista.
Reunificación de guiones	Recompone palabras partidas por guion de fin de línea.
Columnas sin encabezado	Cuatro archivos CSV incluyen una columna anónima con 231.830 líneas. Se verificó que forman secuencias consecutivas con 26 reinicios a cero -exportaciones en lotes concatenadas-, es decir el índice del exporte y no un dato. §2.1 pide además que cada valor conserve el nombre de su columna como contexto, y una columna sin encabezado no lo aporta. Se elimina solo cuando el valor es un número sin texto.

El resultado sobre los 1.826 documentos es una reducción de 8.245.459 caracteres (3,85 %), sin que ningún documento con texto quedara vacío.

Detección de idioma. Se emplea un clasificador estadístico basado en n-gramas de bytes, con licencia BSD y

salida determinista, condición que descarta alternativas que muestrean aleatoriamente y varían entre ejecuciones. No es un modelo generativo. No se restringe la lista de idiomas candidatos, porque el corpus contiene documentos legítimos en árabe, ruso, chino, coreano, japonés y francés; restringirla haría que el campo mintiera precisamente donde más se notaría. El reparto resultante es: inglés 1.004 documentos (55,0 %), español 641 (35,1 %), portugués 108 (5,9 %), otros idiomas 55 (3,0 %) y 18 sin señal suficiente (1,0 %).

El orden de las etapas es significativo: la limpieza precede a la detección. En sentido inverso, la marca de agua de un documento, que ocupa el 69,3 % de su texto extraído, provocaba que un documento en inglés se clasificara como ruso.

2.4 Resultado de la extracción

La ejecución completa produjo 1.826 documentos con cero fallos y 28.881.496 palabras. Ocho documentos quedaron sin texto y su exclusión del índice está justificada: cinco son imágenes fotográficas sin texto que extraer y tres son archivos JSON que no son documentos, sino manifiestos del proceso de descarga con campos como `total_publicaciones` o `scraped_at`. La Tabla 1 define metadata por fragmento, y un documento sin texto no produce ningún fragmento; añadir líneas sin vector asociado rompería el invariante de §5.3.

3. Fragmentación

3.1 Estrategia: cascada híbrida de cuatro niveles

§3.2 admite estrategias híbridas siempre que se justifiquen explícitamente. La implementada recorre cuatro niveles en orden, descendiendo solo cuando el anterior no basta:

- **Nivel 1. Agrupación de bloques.** Los saltos de párrafo del texto normalizado delimitan párrafos en documentos de prosa y filas en documentos tabulares. Se agrupan bloques completos hasta agotar el presupuesto.
- **Nivel 2. Retroceso a límite de oración.** Cuando el bloque excede el presupuesto, el corte retrocede al final de la última oración completa que cabe. Es la formulación literal de §3.3.
- **Nivel 3. Separadores secundarios.** Punto y coma, dos puntos, puntos guía y saltos de línea simples, para unidades que no son oraciones.
- **Nivel 4. Corte por longitud,** como último recurso, siempre entre palabras y nunca dentro de una.

3.2 Justificación frente a una estrategia simple

Se midió, sobre los 1.826 documentos normalizados, cuántos bloques exceden por sí solos el presupuesto:

	Bloques No caben en el presupuesto	
Prosa (1.715 documentos)	57.613	17.218 (29,89 %)
Tabular (103 documentos)	276.925	83 (0,03 %)

Casi tres de cada diez párrafos de prosa se exceden por sí mismos. Una estrategia «por párrafo» habría producido 17.218 fragmentos por encima del límite del encoder, que este truncaría sin emitir aviso alguno. El nivel 2 es por tanto necesario y no una mejora opcional. En la ejecución entregada, los niveles 3 y 4 -los únicos donde la completitud lingüística y el tope de longitud pueden entrar en conflicto- intervienen en el 1,24 % de los fragmentos, y lo hacen sobre celdas de tabla, no sobre prosa.

Un salto de párrafo no es siempre frontera de oración. Los extractores de PDF emiten un salto de bloque también en los cambios de columna y de página, de modo que agrupar bloques completos partía oraciones. Se resuelve uniendo un bloque con el siguiente únicamente cuando el primero no cierra oración y el segundo comienza en minúscula. Las dos condiciones conjuntas distinguen una oración realmente partida de un encabezado o un pie de figura, que legítimamente carecen de punto final pero preceden a algo que empieza en mayúscula.

Se evaluó y descartó el uso de un divisor recursivo de biblioteca. Su lista de separadores por omisión no incluye el punto, por lo que no mantiene las oraciones íntegras e incumple §3.3, que es requisito obligatorio; aunque se le añada, sus últimos recursos parten palabras; cuenta caracteres en lugar de tokens; y no distingue prosa de contenido tabular.

3.3 Tratamiento de los documentos tabulares

Los 103 documentos tabulares (CSV, XLSX y PBF) concentran el 52,1 % de las palabras del corpus desde el 5,6 % de los archivos, y son en su mayoría volcados bibliográficos. §2.1 indica que cada fila «puede» tratarse como unidad de fragmentación independiente; esa lectura literal produciría 276.925 fragmentos tabulares, el 87,7 % del índice.

La granularidad elegida agrupa filas completas hasta agotar el presupuesto, lo que reduce los fragmentos

tabulares un 83,6 % sin perder una sola palabra. El reparto resultante en el índice entregado es de 45.245 fragmentos de prosa (49,7 %) y 45.776 tabulares (50,3 %).

No se consideró excluir estos documentos. §1.3 establece que el conjunto provisto para los tres fenómenos constituye el corpus, y un documento excluido del índice no puede recuperarse en ninguna circunstancia. La decisión quedó validada por el comportamiento observado del sistema: pese a constituir la mitad del índice, los documentos tabulares no aparecen en ninguno de los 500 fragmentos ni de los 150 documentos que devuelve el archivo de resultados. El encoder ordena la prosa por encima de los registros bibliográficos sin necesidad de ningún filtro.

3.4 Verificación

La fragmentación produjo 91.021 fragmentos sobre 1.818 documentos, sin que ningún documento con texto quedara sin fragmentos. Cinco pruebas automáticas se superaron: §3.3, ningún corte parte una oración sobre más de 42.000 cortes de prosa examinados; cobertura, cada documento se reconstruye carácter a carácter concatenando sus fragmentos; Tabla 1, los ocho campos presentes, `posicion` consecutiva desde 0 y 91.021 identificadores únicos; presupuesto; y determinismo, dos ejecuciones producen idénticos identificadores y textos.

La prueba de §3.3 se define sobre el corte y no sobre el fragmento aislado. Preguntar si un fragmento termina en signo de puntuación produce más de doce mil falsos positivos, porque los encabezados y pies de figura carecen legítimamente de punto final. El síntoma inequívoco es la pareja: un fragmento que no cierra seguido de otro que comienza en minúscula.

4. Codificación semántica

4.1 Modelo seleccionado

BAAI/bge-m3, revisión 5617a9f61b028005a4858fdac845db406aefb181, 1.024 dimensiones.

Criterio de §4.3	Verificación
Familia encoder (§4.2)	Arquitectura XLM-RoBERTa. No es un decoder.
Soporte multilingüe	Más de 100 idiomas; español, inglés y portugués nativos.
Alineamiento translingüe	Entrenado con objetivo translingüe explícito y evaluado en MIRACL y MKQA.
Recuperación densa	Diseñado para recuperación, no para clasificación ni similitud de pares, distinción que §4.3 pide establecer y que descarta modelos orientados a minería de textos paralelos.
Licencia	MIT, una de las tres que §4.3 prefiere.
Dimensionalidad	1.024. §4.3 advierte que dimensiones más altas no garantizan mejor rendimiento: no se elige por grande, sino porque es la que produce el modelo.
Eficiencia computacional	Es su punto débil; véase 4.2.

El criterio determinante es el alineamiento translingüe, no el soporte multilingüe. El conjunto de evaluación entregado presenta las cincuenta consultas en español, mientras que el corpus es 55,0 % inglés. La recuperación es por tanto translingüe español-inglés, propiedad más exigente que comprender varios idiomas por separado, y que obliga a atender a las cifras translingües de los benchmarks públicos y no a las monolingües.

Se prefirió este modelo frente a alternativas de dimensión equivalente y licencia igualmente permisiva por una razón operativa adicional: no requiere prefijos de instrucción en la consulta y en el pasaje, cuya omisión en uno de los dos lados degrada la calidad sin producir ningún error.

Se emplea únicamente la modalidad densa. El modelo genera también representaciones dispersas y multi-vector, que no son representables en un índice de un vector por fragmento.

4.2 Coste y límite de tokens

§4.3 enumera la eficiencia computacional entre los criterios de selección. Medida sobre fragmentos reales del corpus en CPU sin GPU, la codificación completa con este modelo requeriría del orden de 363 horas, frente a las 12 horas de un modelo de 384 dimensiones de la misma familia. Se conservó el modelo de mayor calidad translingüe y la codificación se ejecutó en una máquina con GPU, en 2 h 48 min a unos 14 fragmentos por segundo. Esta decisión no afecta a la reproducibilidad que evalúa §1.4, que consiste en que `generador.py` reproduzca los resultados a partir del índice ya construido.

§4.3 indica que los modelos encoder tienen un límite de tokens de entrada, «comúnmente 512», y que los

fragmentos deben diseñarse para no superar dicho límite. El límite del modelo empleado es de **8.192 tokens**, comprobado en tiempo de ejecución. La distribución real de `num_tokens` en el índice entregado, medida con el tokenizador del propio modelo, es:

Mediana	Percentil 95	Máximo	Fragmentos sobre 512	Fragmentos sobre 8.192
692 tokens	930 tokens	6.867 tokens	68,24 %	0

Ningún fragmento se trunca; el máximo real se sitúa un 16 % por debajo del límite efectivo. Se declara explícitamente esta cifra porque el valor de 512 que §4.3 cita como habitual no es el aplicable a este encoder. La relación entre tokens y palabras en este corpus, 2,145, es sensiblemente superior a la de un corpus de prosa homogénea: los documentos tabulares, mitad del índice, contienen identificadores y códigos que el tokenizador fragmenta con densidad muy superior a la del texto corrido.

Se contempló el uso de un segundo encoder con fusión de rankings por CombSUM o CombMNZ, previsto en §4.4 y §8.4. Se descartó: al fusionarse por fragmento, el mismo `chunk_id` debe existir en ambos índices y los fragmentos deben respetar el límite del encoder más restrictivo. Con el 68,24 % de los fragmentos por encima de 512 tokens, un segundo encoder con ese tope truncaría dos tercios del índice sin emitir aviso, y su incorporación exigiría volver a fragmentar y recodificar desde cero.

4.3 Verificación de la matriz

La matriz resultante se verificó sobre la totalidad de las filas y no sobre una muestra: forma 91.021 x 1.024 en `float32`, cero filas nulas, cero valores NaN o infinitos, y normas de 1,000000 con desviación máxima de 1,19e-07. La normalización es condición necesaria para que el producto interno del índice sea el coseno (§5.2, ecuación 4). Como prueba de discriminación del espacio vectorial, el coseno entre 200 vectores repartidos por todo el índice arroja media 0,485, mínimo 0,232 y máximo 1,000: el espacio discrimina y no ha colapsado.

5. Índice vectorial y almacén de metadata

IndexFlatIP con vectores normalizados, serializado con `faiss.write_index()`.

Decisión	Justificación
IndexFlatIP	§5.2 lo recomienda para el volumen esperado en el reto. Con 91.021 vectores una consulta se resuelve en milisegundos y el conjunto de evaluación son 50 consultas: sustituir búsqueda exacta por aproximada introduciría error en la métrica a cambio de una velocidad innecesaria.
Vectores normalizados	Convierte el producto interno en similitud coseno exacta.
Sin <code>IndexIDMap</code>	Los identificadores internos, asignados por orden de inserción, ya coinciden con los números de línea del almacén de metadata. La indirección resolvería un problema inexistente.

Dimensiones finales: 91.021 vectores de 1.024 dimensiones, 372.822.061 bytes.

El `metadata.jsonl` contiene una línea JSON por fragmento, con los ocho campos obligatorios de la Tabla 1 nombrados en español y dos campos adicionales de los que §3.4 autoriza: `idioma`, porque §8.7 contempla el filtrado por este campo y el módulo de recuperación lo utiliza; y `faiss_id`, el número de línea, que convierte el invariante de §5.3 en una propiedad comprobable en lugar de una suposición. Se descartaron otros candidatos: cada campo adicional se paga 91.021 veces y ninguna etapa del sistema los consumía. El archivo ocupa 240.054.195 bytes en 91.021 líneas.

El campo `fuelle` registra la ruta relativa completa del archivo original y no solo su nombre, porque el corpus contiene 59 nombres de archivo repetidos que afectan a 186 documentos. El emparejamiento con el conjunto de referencia se realiza mediante el `doc_id`, conforme a la aclaración oficial del comité, y ese identificador es el `DOC_ID` del inventario de ADL: se verificó que los 1.818 identificadores del almacén y los 195 que aparecen en el archivo de resultados existen todos en dicho inventario.

6. Recuperación y construcción de la salida

El módulo transforma una consulta en tres documentos y diez fragmentos mediante codificación de la consulta, búsqueda en el índice, ajuste de puntuaciones, agregación a nivel documento, filtrado y división al límite de palabras. Toda la operación se realiza sobre vectores, puntuaciones y metadata.

Se recuperan 200 candidatos y no diez. Entre la búsqueda y la respuesta operan tres filtros que descartan candidatos, y con un índice plano el coste de solicitar más es marginal, porque el trabajo consiste en recorrer los 91.021 vectores en cualquier caso. Cuando los candidatos no contienen al menos tres documentos distintos, la búsqueda amplía progresivamente su número: §9.3.1 penaliza o descarta los objetos cuyos arrays no midan exactamente 3 y 10 elementos.

La relevancia de un documento se agrega por máximo de las puntuaciones de sus fragmentos. §8.6 admite máximo, suma o media. La suma es inadecuada en este corpus: un único archivo tabular aporta 33.396 fragmentos frente a una mediana de 5 por documento, de modo que sumar premiaría el tamaño y los tres puestos de la respuesta los ocuparían invariablemente los mismos archivos.

El almacén de metadata se consulta por desplazamiento de bytes, manteniendo en memoria un índice de 91.021 posiciones en lugar de los 240 MB del archivo completo.

6.1 Ajustes de puntuación

Los tres ajustes comparten un criterio: **el fallo de un filtro es irreversible y el de una bonificación no**. §8.7 permitiría filtrar por metadata, pero un documento relevante descartado por un filtro queda inalcanzable y la métrica lo contabiliza como fallo sin remedio.

Tope de dos fragmentos por documento. Sin él, una consulta puede consumir siete de sus diez posiciones con fragmentos de un mismo documento. El tope es una preferencia: si tras aplicarlo no se alcanzan diez fragmentos, se realiza una segunda pasada sin restricción, porque cumplir §9.3.1 es obligatorio.

Deduplicación por contenido. El índice contiene 91.021 fragmentos pero 87.424 textos únicos: 3.597 (4,0 %) repiten contenido, con un grupo mayor de 11 copias, en dos patrones -filas tabulares idénticas repartidas entre documentos y secciones repetidas dentro de un mismo informe-. Puesto que §10.2.1 evalúa la relevancia sobre el campo `text`, dos fragmentos idénticos ocuparían dos posiciones con la misma información. La clave de comparación normaliza únicamente el espaciado; no se normalizan mayúsculas ni puntuación, porque dos textos que difieren en ellas difieren realmente.

Bonificación por fenómeno: factor 1,03. La proporción de documentos devueltos pertenecientes al fenómeno de la consulta era del 80,7 %, con el fenómeno 1 en el 60,4 %. La causa está identificada: las etiquetas de fenómeno describen el observatorio de origen y no el contenido, y existen fuentes catalogadas en un fenómeno que publican abundantemente sobre el tema de otro. El factor se determinó comparando cuatro configuraciones mediante *pooling* -metodología TREC- sobre juicios de relevancia elaborados manualmente, puntuados con las métricas de §10:

Factor	Correspondencia con el fenómeno	Documentos de otro fenómeno que sobreviven	F1@3 estricto	F1@3 permisivo
1,00	60,4 %	29	-	-
1,03	81,2 %	13	0,595	0,905
1,05	93,8 %	5	0,338	0,857
1,10	100,0 %	0	0,338	0,810

Con 1,10 no sobrevive ningún documento de otro fenómeno: equivale a un filtro. Aplicado el factor 1,03, la correspondencia medida sobre el archivo entregado asciende al 91,3 %.

Factor por idioma: 0,97 para idiomas distintos de español e inglés. Varias organizaciones del corpus publican el mismo informe en varios idiomas, que constituyen documentos distintos con identificadores distintos. Una consulta podía dedicar posiciones a versiones traducidas del mismo contenido y, en un caso, ocupar dos de sus tres puestos de documento con dos traducciones del mismo resumen ejecutivo existiendo el informe completo en el corpus. El factor se determinó por barrido: con 1,00 sobreviven 16 de 500 fragmentos fuera de ese par de idiomas en 8 consultas; con 0,99 el conjunto de documentos solo cambia en 2 de las 50 consultas, y §10.2.2 establece que F1@3 no considera el orden; con 0,95 y valores inferiores no sobrevive ninguno, lo que constituye un filtro encubierto. El valor 0,97 corresponde a una banda de tolerancia del 3 % en similitud coseno: solo se prefiere la versión en español o inglés cuando ambas son semánticamente casi equivalentes. Los documentos sin idioma detectado no se penalizan, porque la ausencia de etiqueta indica falta de señal del clasificador y no un idioma distinto.

6.2 Límite de 250 palabras

El 90,6 % de los fragmentos del índice excede las 250 palabras, de modo que la división es el caso normal y no la excepción. El límite se aplica al construir la salida, no al índice: los fragmentos extensos producen vectores con más contexto y se dividen únicamente al reportarse. La división respeta la misma completitud lingüística de §3.3, reutilizando el divisor de oraciones de la fragmentación para evitar dos implementaciones divergentes de la misma regla. Verificado sobre el índice completo: 175.025 subfragmentos, mediana de 169 palabras, máximo 250, ninguno por encima del límite y ningún fragmento que pierda texto.

Se reporta un subfragmento por fragmento, no todos. §9.2.1 prohíbe situar dos en una misma posición del ranking, pero no obliga a reportarlos todos; reportándolos, las diez posiciones procederían de unos cinco fragmentos distintos y las porciones sin contenido relevante ocuparían posiciones altas, que es precisamente lo

que penaliza NDCG@10. La selección se realiza por solapamiento léxico con la consulta y no por similitud vectorial: codificar los subfragmentos en tiempo de consulta trasladaría un coste de horas a quien reproduce los resultados, mientras que el solapamiento es determinista, cuesta microsegundos y no constituye un modelo.

6.3 Composición de la salida

El archivo contiene 50 líneas en el orden q001-q050, con exactamente tres documentos y diez fragmentos por consulta y los campos de la Tabla 2. Medido sobre él: los 150 documentos son 132 PDF y 18 JSON, sin ningún documento tabular; el 91,3 % pertenece al fenómeno de su consulta y ninguna consulta carece de documentos de su propio fenómeno; y los 500 fragmentos se reparten en 336 en inglés, 159 en español y 5 en otros idiomas, lo que confirma que la recuperación translingüe opera como se pretendía.

7. Grafo de conocimiento

Componente opcional de §7. Se entrega como `grafo.graphml` dentro de `base_vectorial/grafo/`, con 3.375 nodos y 3.460 aristas dirigidas y tipadas.

Reconocimiento de entidades. Se emplea `urchade/gliner_multi-v2.1`, revisión 443d26d6, licencia Apache 2.0. Es un encoder bidireccional construido sobre `mDeBERTa-v3`, de modo que no incurre en la restricción de §4.2, y su naturaleza *zero-shot* permite obtener tipos que un reconocedor clásico no produciría, como tecnologías o normas. Se definieron nueve tipos de entidad con umbral de confianza 0,5. La unidad de entrada son subventanas de los fragmentos del índice con presupuesto de 320 tokens: los fragmentos tienen mediana de 692 tokens y la ventana del modelo es de 384, de modo que pasarle fragmentos completos habría dejado sin analizar la segunda mitad de la mayoría sin ningún aviso. Se procesaron 127.070 subventanas de 48.087 fragmentos en 1 h 57 min sobre GPU T4, sin ninguna trunca.

Extracción de relaciones. Se realiza mediante patrones lingüísticos sobre el texto que media entre dos entidades del mismo fragmento, vía que §7.2 autoriza expresamente junto a los modelos de clasificación y las dependencias sintácticas. No se emplea ningún modelo generativo.

Canonicalización. Cada entidad se reduce a una clave determinista y se aplican alias curados manualmente, de modo que las variantes de una misma organización o país converjan en un único nodo.

Trazabilidad. Cada arista conserva el `doc_id` y el `chunk_id` de origen, conforme a §7.2, lo que permite rastrear la evidencia textual de cada relación. Los 3.209 identificadores de fragmento que aparecen en el grafo existen en el `metadata.jsonl`. Aportan evidencia 484 de los 1.818 documentos (26,6 %), repartidos de forma equilibrada entre los tres fenómenos: 151 de F1, 181 de F2 y 152 de F3.

El archivo se somete a siete pruebas automáticas que verifican estructura, trazabilidad, canonicalización, integridad al releerse, cobertura, ausencia de dependencias generativas e interoperabilidad. Esta última se añadió tras detectar que un archivo que superaba las seis anteriores no abría en ningún visor: las aristas compartían trece valores de identificador, y GraphML los exige globalmente únicos. Ninguna prueba lo detectaba porque la biblioteca de grafos interpreta ese identificador como clave por par de nodos, de modo que toda comprobación que partiera del grafo ya cargado era ciega por construcción. La prueba añadida lee el XML en crudo.

El grafo no se integró en el flujo de recuperación. §8.5 lo plantea como posibilidad y no como requisito; integrarlo modificaría el archivo de resultados y obligaría a repetir la verificación de reproducibilidad, y su efecto sobre las métricas no es medible sin el conjunto de referencia.

8. Reproducibilidad

`generador.py` cumple tres condiciones. No importa ninguna dependencia de extracción ni de OCR, porque el resultado de esas etapas está contenido en el índice y arrastrarlas al entorno de evaluación solo añadiría formas de fallo. La revisión del modelo está fijada por su identificador de commit y no solo por su nombre: sin ello podría descargarse una versión distinta, los vectores de la consulta dejarían de residir en el mismo espacio que los del índice y el ranking cambiaría sin producir error. Y los empates de puntuación se resuelven de forma determinista por `chunk_id`, sin depender del orden que devuelva la biblioteca de búsqueda.

Cada ejecución registra un manifiesto con la configuración de recuperación aplicada, las versiones de las bibliotecas, el modelo con su revisión y las sumas SHA-256 del índice y de la salida. El modo `--comprobar` contrasta ese registro con el código en ejecución en segundos y sin descargar el modelo, y señala cualquier divergencia. Existe porque un archivo de resultados válido y un código válido pueden no corresponderse entre sí, y ninguna prueba de formato lo detectaría.

La reproducibilidad se comprobó eliminando el archivo de resultados y regenerándolo desde cero en dos

sesiones independientes, con descarga fresca del modelo y versiones distintas de biblioteca, obteniendo un archivo idéntico byte a byte: 801.865 bytes, 50 líneas, SHA-256 `1ec6f8200bd7...`. El entorno de referencia queda documentado en el manifiesto.

Reproducir los resultados requiere nueve paquetes de Python declarados en `requirements.txt`: cuatro importados directamente y cinco transitivos fijados por determinar los valores numéricos de los embeddings. Se declara `faiss-cpu` y no `faiss-gpu`, dado que una vez construido el índice la ejecución en CPU es suficiente. El conjunto es deliberadamente reducido: cada dependencia adicional es una vía más para que la instalación falle.

La construcción del índice desde el corpus -no la reproducción de los resultados- requiere además **Tesseract OCR 5.4.0** con los idiomas español e inglés, dependencia de sistema que los gestores de paquetes de Python no capturan. La ejecución que generó el archivo entregado se realizó en un entorno sin Tesseract y sin bibliotecas de extracción de PDF instaladas, lo que confirma que no interviene en la reproducción.

Estrategia de verificación. Cada etapa dispone de un verificador independiente que devuelve código de salida distinto de cero ante cualquier fallo: 5 comprobaciones sobre la normalización, 5 sobre la fragmentación, 5 sobre el índice, 6 sobre el archivo de resultados, 7 sobre el grafo y 12 sobre la implementación de las métricas de §10, todas superadas. Un criterio metodológico gobierna estas pruebas: una verificación que toma su entrada de la misma fuente que valida no verifica nada. La comprobación del invariante de §5.3 no interroga al índice sobre su propio contenido, sino que parte de la matriz de embeddings que produjo el encoder.

9. Limitaciones

- Las métricas de §10 no se han evaluado contra el conjunto de referencia, que no es público durante el reto. Las comparaciones realizadas para elegir los parámetros de recuperación emplean *pooling* sobre juicios de relevancia elaborados manualmente: son válidas para comparar configuraciones entre sí, pero sus valores absolutos no son comparables con los de la evaluación oficial.
- La bonificación por fenómeno presupone que el conjunto de referencia se construyó dentro del fenómeno de cada consulta. Si se construyó por tema, el ajuste sería contraproducente. No es verificable con la información disponible.
- Los documentos tabulares constituyen la mitad del índice y son en su mayoría volcados bibliográficos. Su inclusión es obligada por §1.3 y la granularidad elegida reduce su peso sin excluirlos.
- La detección de idioma presenta un error estimado del 1 %, concentrado en documentos cuya extracción es defectuosa. Por ese motivo el idioma se emplea como factor de puntuación y nunca como filtro.
- Un pequeño número de PDF presenta extracción degradada, con palabras unidas sin separador o letras duplicadas, por ausencia de caracteres separadores en el archivo de origen. No se cuantificó su alcance y no admite corrección sin un segmentador de palabras.
- El determinismo tiene un límite medido: cuatro consultas presentan empates prácticos en su lista de candidatos, con márgenes del orden de $1e-08$, que el desempate por identificador no cubre porque solo actúa sobre empates exactos. La vía que ejecuta quien reproduce la entrega resultó estable en las dos sesiones comprobadas.

10. Estructura de la entrega

entrega/	
resultados.jsonl	50 líneas, una por consulta
resultados.manifest.json	configuración, versiones y hashes de la corrida
generador.py	reproduce resultados.jsonl desde el índice
requirements.txt	9 paquetes
informe_tecnico.pdf	este documento
lib/	modulos que consume generador.py
base_vectorial/	
encoder_bge-m3/	
index.faiss	IndexFlatIP, 91.021 x 1.024
metadata.jsonl	Tabla 1 mas idioma y faiss_id, 91.021 líneas
grafo/	
grafo.graphml	3.375 nodos, 3.460 aristas

Ejecución: `pip install -r requirements.txt, python generador.py y python generador.py --comprobar` para verificar la reproducibilidad.